# Artificial Neural Networks

# Assignment 1

Saad Karim
Roll Number: 22P-9277
Section: AI-6A

Submitted to: Dr. Omar Usman

# Contents

**Task 0: Getting Ready with your Dataset** **2**

**Task 1: Relationship between Learning Rate and Iterations** **3**

**Task 2: Converting to Stochastic Gradient Descent** **7**

**Task 3: Fitting to Polynomial Regression** **16**

**Deliverables** **20**

# Task 0: Getting Ready with your Dataset

## Question 1

Which columns are most suitable to provide a linear/multilinear/polynomial relationship to price?

**Answer: The most suitable columns for a linear relationship are Carats (strongest) and possibly x, y, z (but redundant with Carats). For a multilinear relationship, use Carats, Cut, Color, Clarity, Depth, Table together, excluding x, y, z to avoid multicollinearity. For a polynomial relationship, Carats (or x, y, z) is best, likely with a quadratic or cubic model to capture non-linear price growth.**

## Question 2

Why are some plots appearing as vertical or horizontal lines?

**Answer: Vertical lines appear in plots of Cut, Color, or Clarity vs. Price because these features are ordinal, with discrete values (e.g., 1, 2, 3), causing prices to align vertically at each value. Horizontal lines are unlikely but could arise from repeated price values or data anomalies across continuous features like Carats or Depth.**

# Task 1: Relationship between Learning Rate and Iterations

## Question 1

How many iterations would be needed for Learning Rate 0.01 to match 0.1?

**Answer: Approximately 100 iterations are needed for LR = 0.01 to make its linear regression line similar to LR = 0.1 after 10 iterations. This is because LR = 0.01 takes steps 10 times smaller, requiring roughly 10 times more iterations to converge to a similar point.**

## Question 2

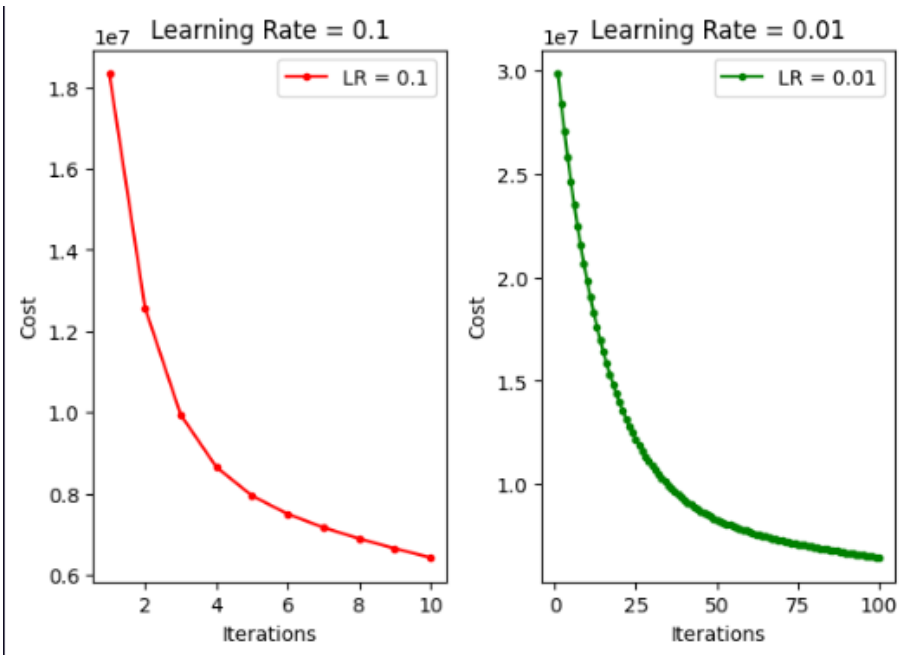
Cost function implementation and minimum error comparison.

**a) Code:**

```
for idx, (learning_rate, color, num_iterations) in enumerate(zip([0.1, 0.01], ['r', '
    b = 0
    m = 0
    costs = []

    for i in range(num_iterations):
        m_gradient = 0
        b_gradient = 0
        N = float(len(points))
        for j in range(0, len(points)):
            m_gradient += (2 / N) * x[j] * (y[j] - (m * x[j] + b))
            b_gradient += (2 / N) * (y[j] - (m * x[j] + b))
        m = m + learning_rate * m_gradient
        b = b + learning_rate * b_gradient
        cost = np.sum((y - (m * x + b)) ** 2) / N
        costs.append(cost)

    plt.subplot(1, 2, idx + 1)
    plt.plot(range(1, num_iterations + 1), costs, marker='.', color=color, label=f'LR
    plt.xlabel('Iterations')
    plt.ylabel('Cost')
    plt.title(f'Learning Rate = {learning_rate}')
    plt.legend()

plt.tight_layout()
plt.show()
```

```
# Q2, part b
for learning_rate, num_iterations in zip([0.1, 0.01], [10, 100]):
    b = 0
    m = 0
    costs = []  # To store cost for each iteration

    for i in range(num_iterations):
        m_gradient = 0
        b_gradient = 0
        N = float(len(points))
        # Compute gradients
        for j in range(0, len(points)):
            m_gradient += (2 / N) * x[j] * (y[j] - (m * x[j] + b))
            b_gradient += (2 / N) * (y[j] - (m * x[j] + b))
        m = m + learning_rate * m_gradient
        b = b + learning_rate * b_gradient

        # Compute cost (MSE)
        cost = np.sum((y - (m * x + b)) ** 2) / N
        costs.append(cost)
        print(f'LR = {learning_rate}, Iteration {i+1}, Cost: {cost}')

    print(f'Minimum cost for LR = {learning_rate}: {min(costs)}')
```

**b) Answer:**



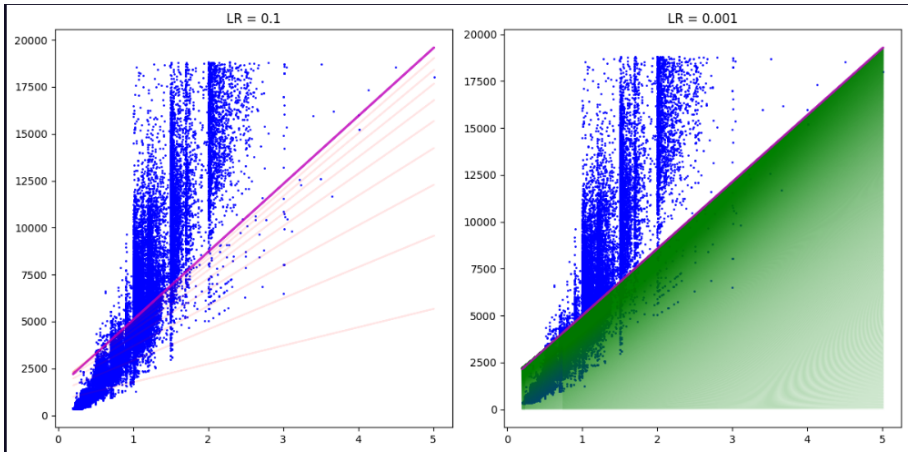

Minimum cost for LR = 0.1: 6424415.936567521
Minimum cost for LR = 0.01: 6464094.121247796

## Question 3

Modify code for Learning Rate 0.001 to match output of 0.1. Try 0.0001?

**a) Code:**

```
# Q3, and last part
learning_rates = [0.1, 0.001]
colors = ['r', 'g']

plt.figure(figsize=(12, 6))

for idx, (learning_rate, color, num_iterations) in enumerate(zip(learning_rates, colo
    b = 0
    m = 0
    plt.subplot(1, 2, idx + 1)
    plt.scatter(x, y, s=1, c='b')
    for i in range(num_iterations):
        m_gradient = 0
        b_gradient = 0
        N = float(len(points))
        for j in range(0, len(points)):
            m_gradient += (2 / N) * x[j] * (y[j] - (m * x[j] + b))
            b_gradient += (2 / N) * (y[j] - (m * x[j] + b))

        m = m + learning_rate * m_gradient
        b = b + learning_rate * b_gradient
        pred = m * x + b
        plt.plot(x, pred, c=color, alpha=0.1)

    plt.plot(x, pred, c='m', alpha=0.8, linewidth=2)
    plt.title(f'LR = {learning_rate}')

plt.tight_layout()
plt.show()
```




## Question 4

Behavior of regression line—smooth or oscillating? Would you be willing to try a learning rate of 0.0001?

**Answer:The behavior of the regression line is oscillating. Yes, I would be willing to try a learning rate of 0.0001. However, to get a similar result, you would need to increase the number of iterations even further (e.g., to 10,000 or more). Very small learning rates require many more iterations to converge.**

# Task 2: Converting to Stochastic Gradient Descent

## Question 1

Is the line progressing smoothly or oscillating?

**Answer: The linear regression line exhibits oscillation.**

## Question 2

Stochastic gradient solution for multiple learning rates.

**Answer:**

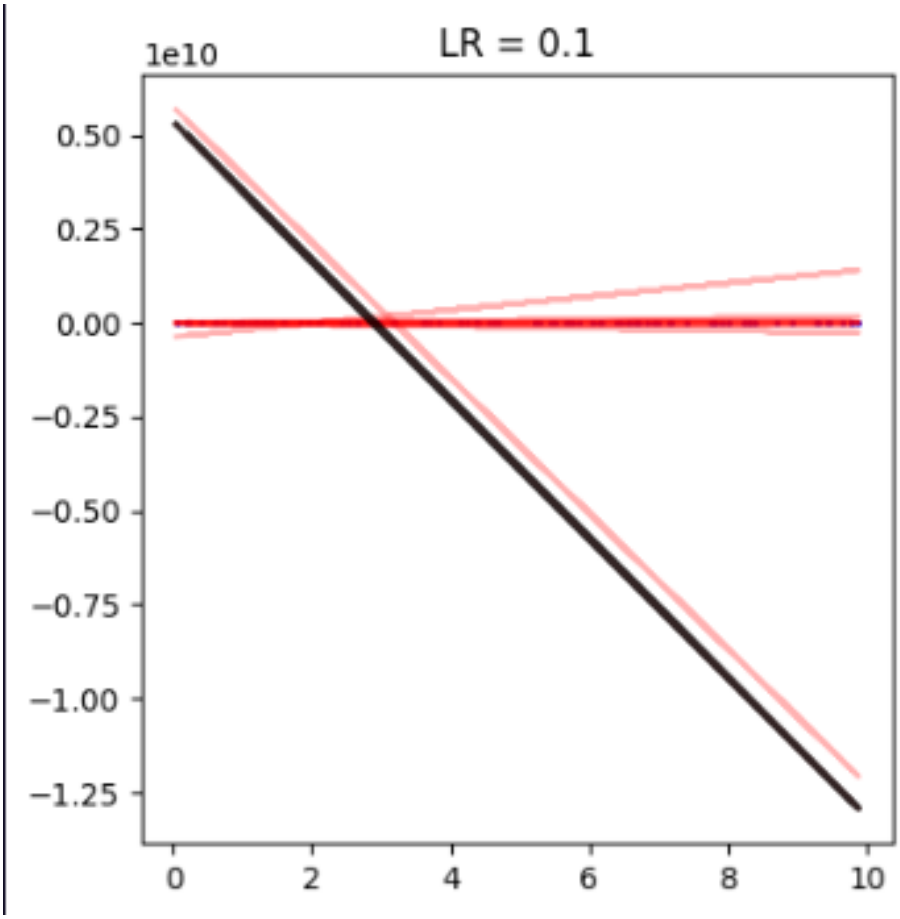
```
np.random.seed(0)
x = np.random.rand(100) * 10
y = 2 * x + 1 + np.random.randn(100) * 2
points = np.array(list(zip(x, y)))
learning_rates = [0.1, 0.01, 0.001, 0.0001, 0.00001]
colors = ['r', 'g', 'b', 'c', 'm']

plt.figure(figsize=(15, 10))

for idx, (learning_rate, color) in enumerate(zip(learning_rates, colors)):
    b = 0
    m = 0
    plt.subplot(2, 3, idx + 1)
    plt.scatter(x, y, s=1, c='b')
    plt.title(f'LR = {learning_rate}')
    cost_graph = []
    for i in range(10):
        m_gradient = 0
        b_gradient = 0
        N = float(len(points))

        idx1 = np.random.randint(0, len(points))
        x_j1, y_j1 = x[idx1], y[idx1]
        m_gradient += 2 * x_j1 * (y_j1 - (m * x_j1 + b))
        m = m + learning_rate * m_gradient

        idx2 = np.random.randint(0, len(points))
        x_j2, y_j2 = x[idx2], y[idx2]
        m_gradient += 2 * x_j2 * (y_j2 - (m * x_j2 + b))
```

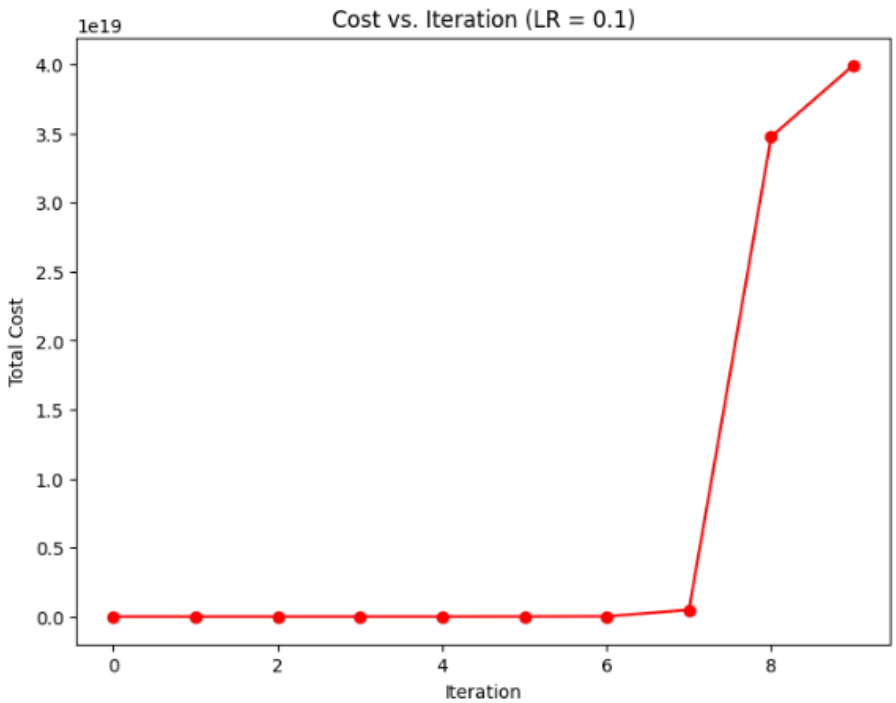
```
        m = m + learning_rate * m_gradient

        b_gradient = 0
        idx1 = np.random.randint(0, len(points))
        x_j1, y_j1 = x[idx1], y[idx1]
        b_gradient +=  2 * (y_j1 - (m * x_j1 + b))
        b = b + learning_rate * b_gradient

        idx2 = np.random.randint(0, len(points))
        x_j2, y_j2 = x[idx2], y[idx2]
        b_gradient +=  2 * (y_j2 - (m * x_j2 + b))
        b = b + learning_rate * b_gradient

        pred = m * x + b
        plt.plot(x, pred, c=color, alpha=0.3)

        total_cost = np.mean((y - (m * x + b)) ** 2)
        cost_graph.append(total_cost)

    plt.plot(x, pred, c='k', alpha=0.8, linewidth=2)
    plt.figure(figsize=(8, 6))
    plt.plot(range(10), cost_graph, marker='o', color=color)
    plt.xlabel('Iteration')
    plt.ylabel('Total Cost')
    plt.title(f'Cost vs. Iteration (LR = {learning_rate})')
    plt.show()

plt.tight_layout()
plt.show()
```

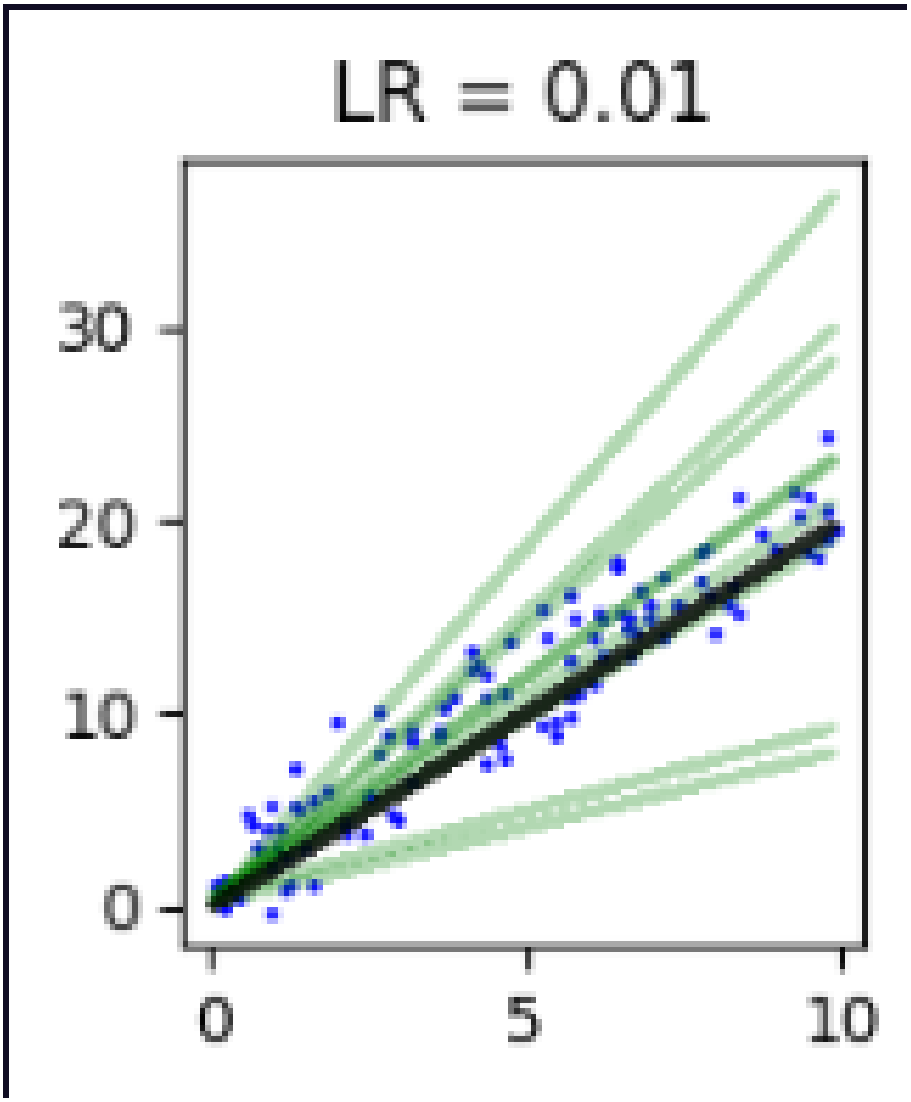
## Question 3

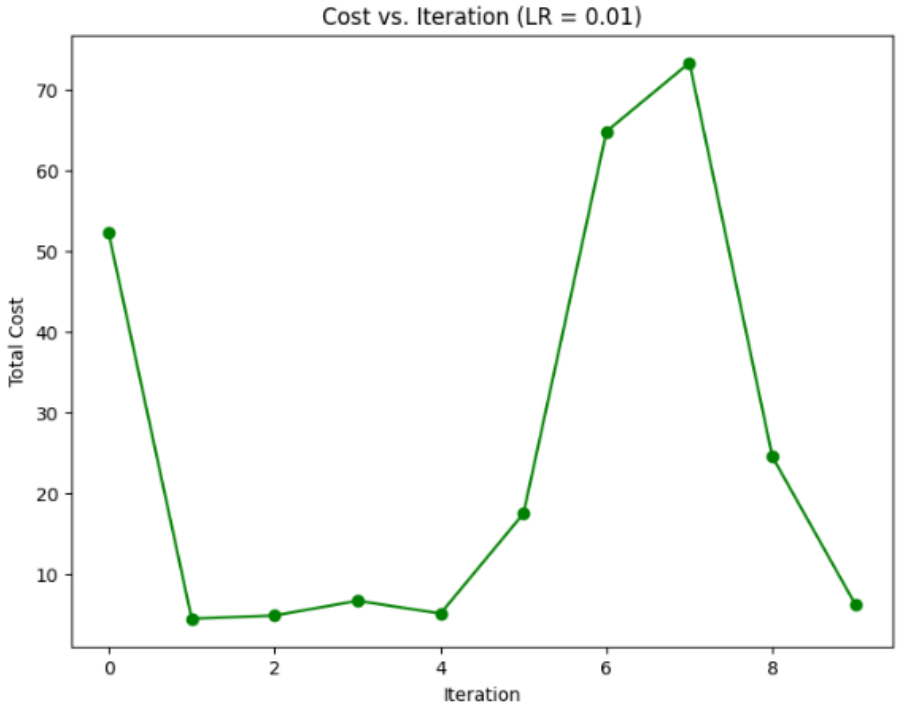
Cost graph for all learning rates.

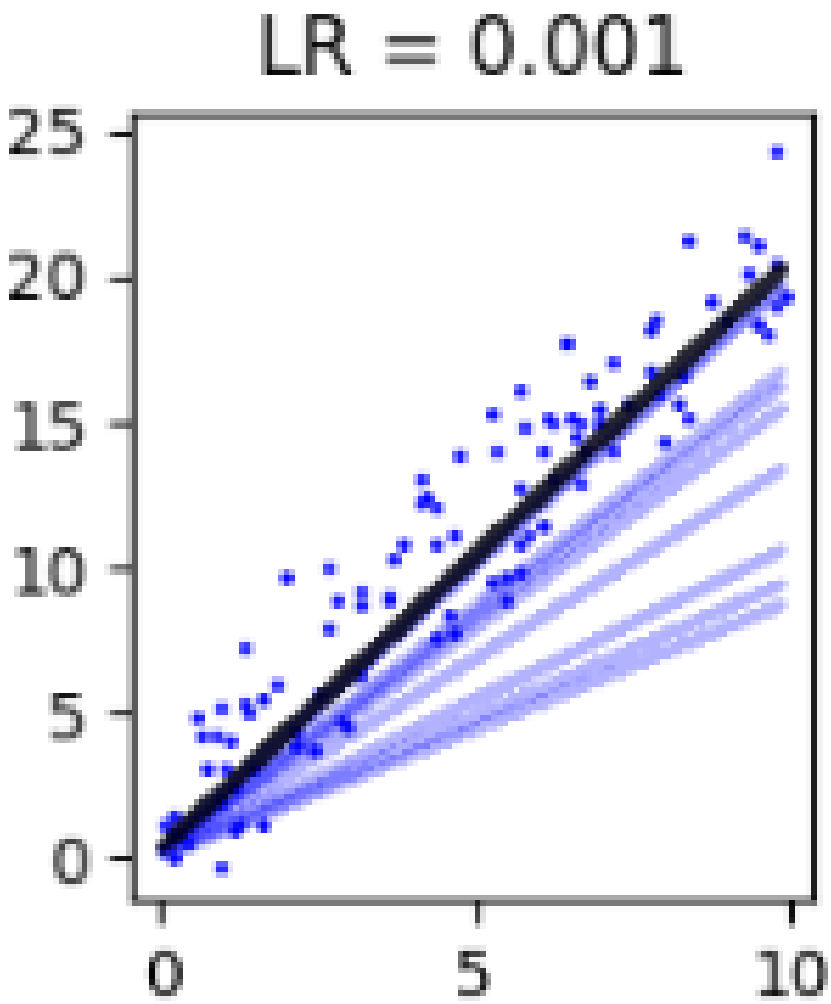
**Graphs:**

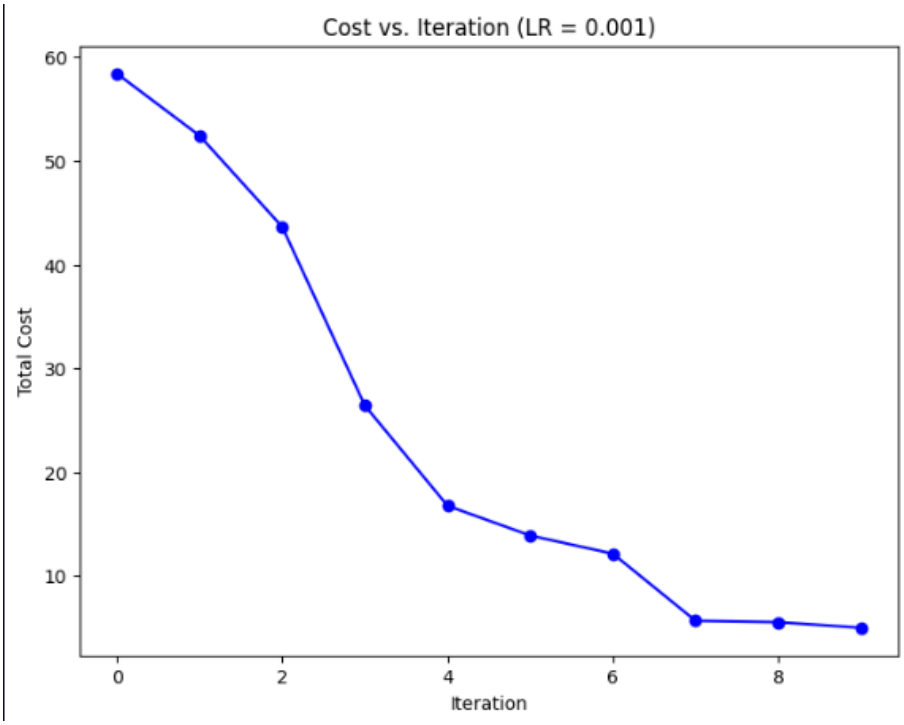

LR = 0.1
1e10
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
0
2
4
6
8
10

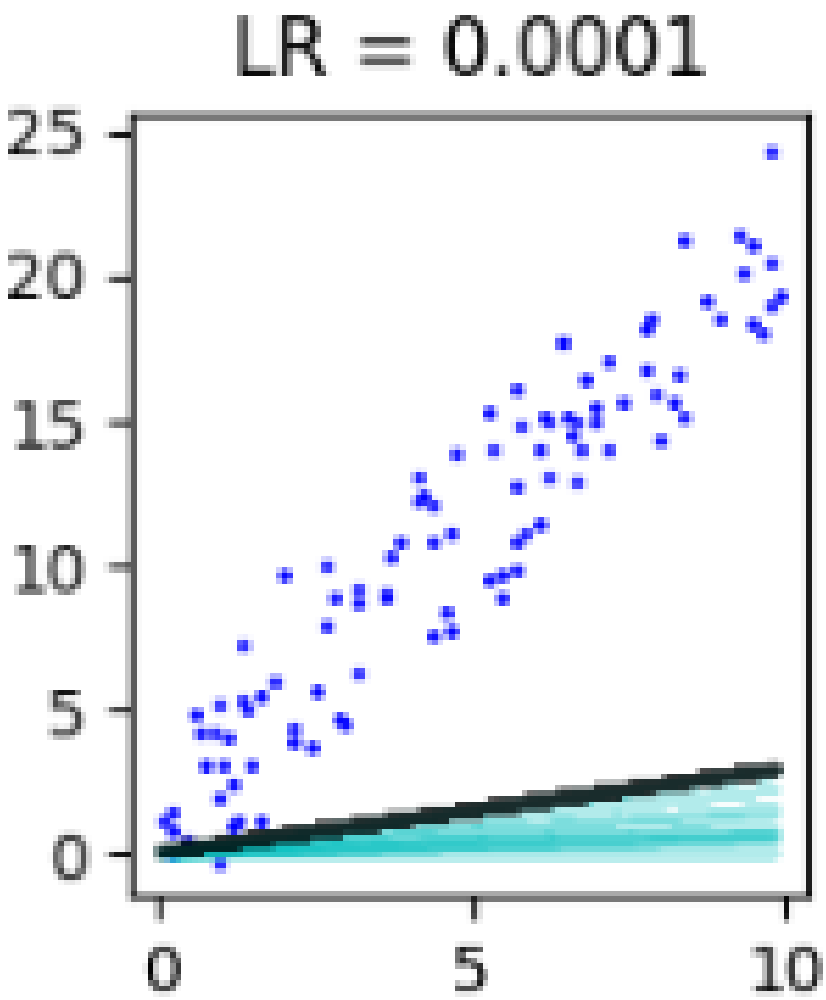

Cost vs. Iteration (LR = 0.1)
1e19
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Total Cost
0
2
4
6
8
Iteration

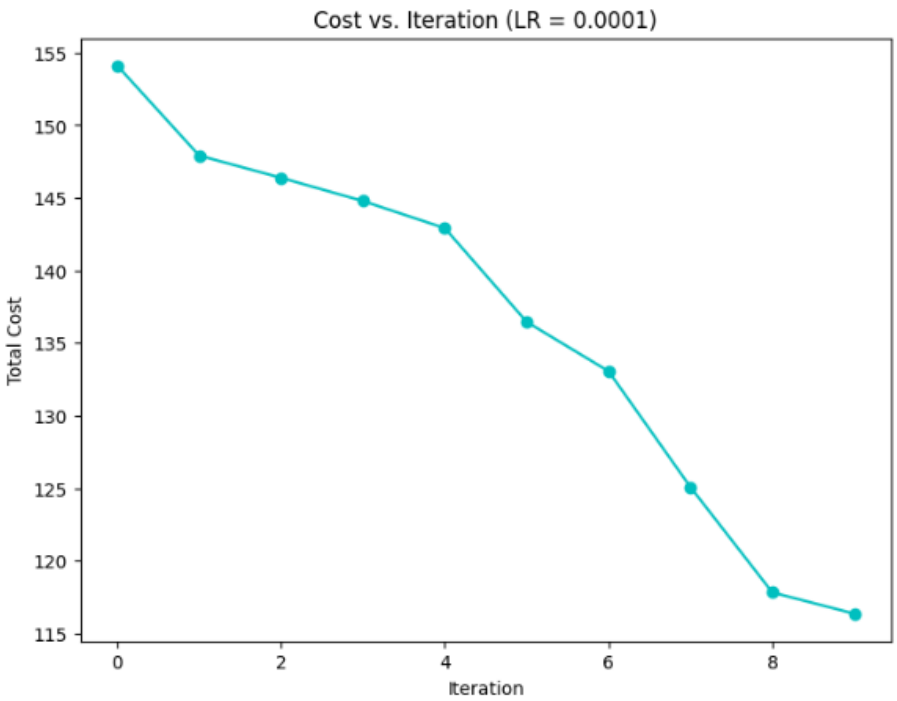

LR = 0.01
30
20
10
0
0
5
10


Cost vs. Iteration (LR = 0.01)
Total Cost
Iteration
70
60
50
40
30
20
10
0
2
4
6
8


LR = 0.001
25
20
15
10
5
0
0
5
10


Cost vs. Iteration (LR = 0.001)
Total Cost
Iteration
60
50
40
30
20
10
0
2
4
6
8


LR = 0.0001
25
20
15
10
5
0
0
5
10


Cost vs. Iteration (LR = 0.0001)
Total Cost
155
150
145
140
135
130
125
120
115
0
2
4
6
8
Iteration

## Question 4

Why are costs not smooth and oscillating?

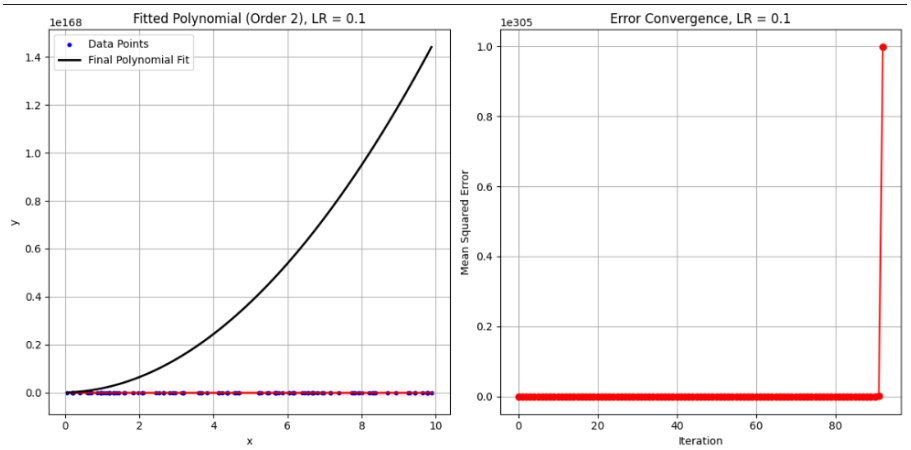
**Answer: Q4: Stochastic Gradient Descent (SGD) uses single random data points for updates, causing noisy gradient estimates. These noisy updates make the parameters, and thus the cost, oscillate. The oscillations arise from the inherent randomness in how SGD samples data.**

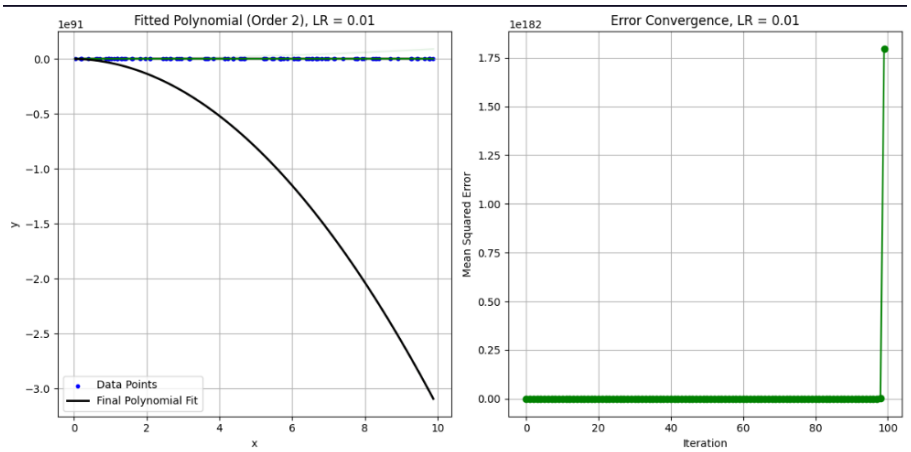
# Task 3: Fitting to Polynomial Regression

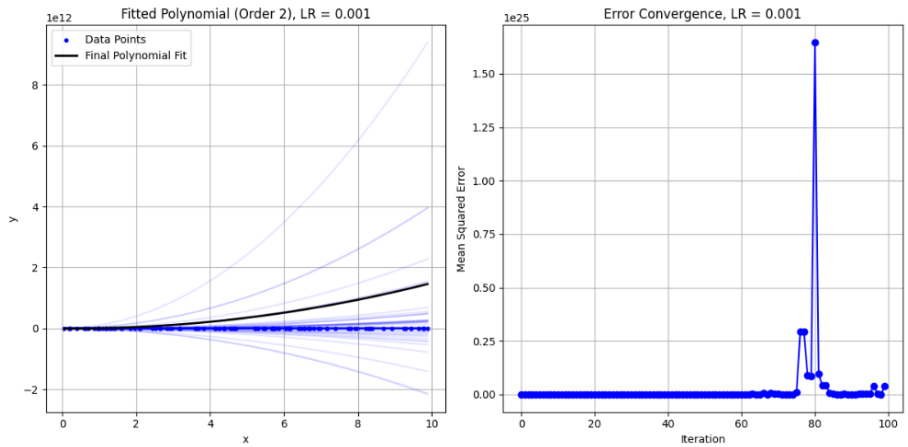
## Question 1

Error convergence and fitted polynomial line (Order 2).

**Answer and Graphs:**



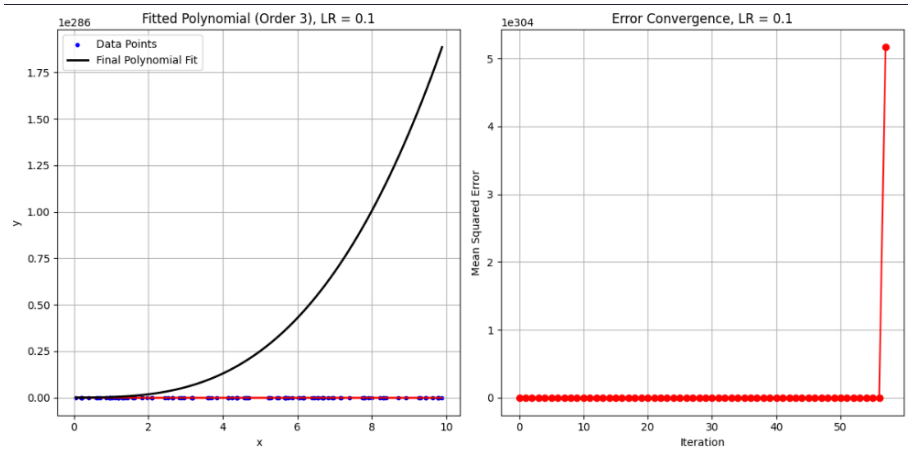



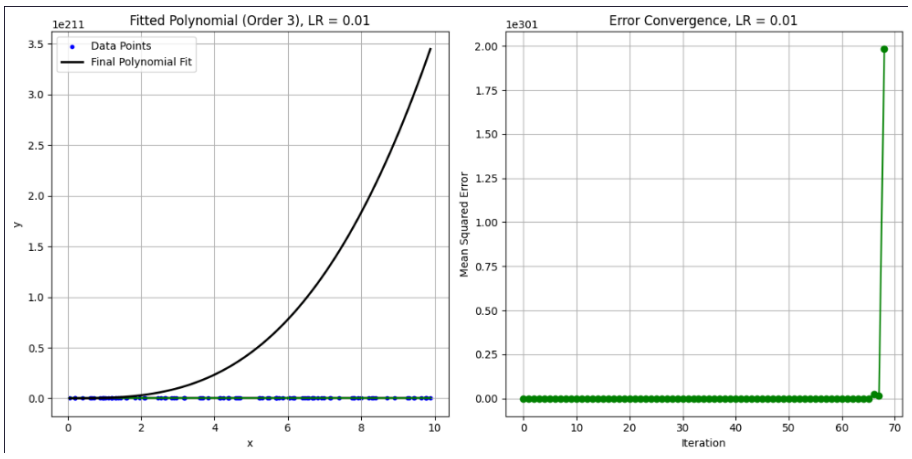



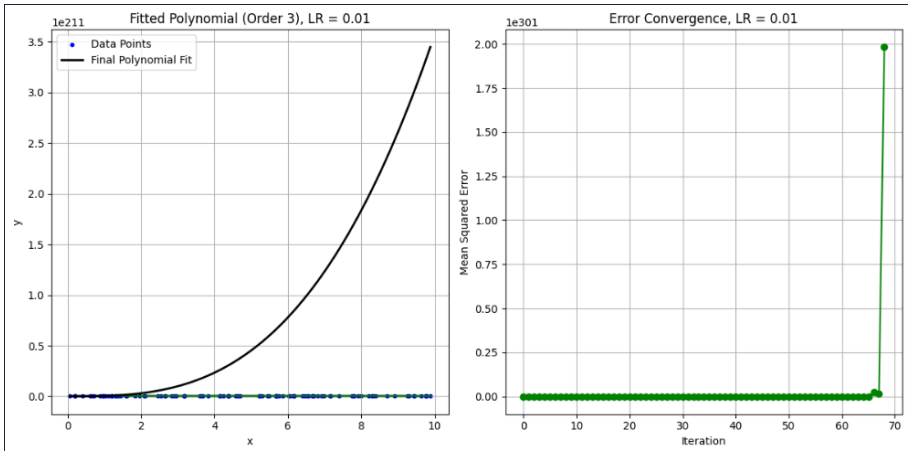
## Question 2

Order 3 polynomial regression. Error + line.

**Answer and Graphs:**



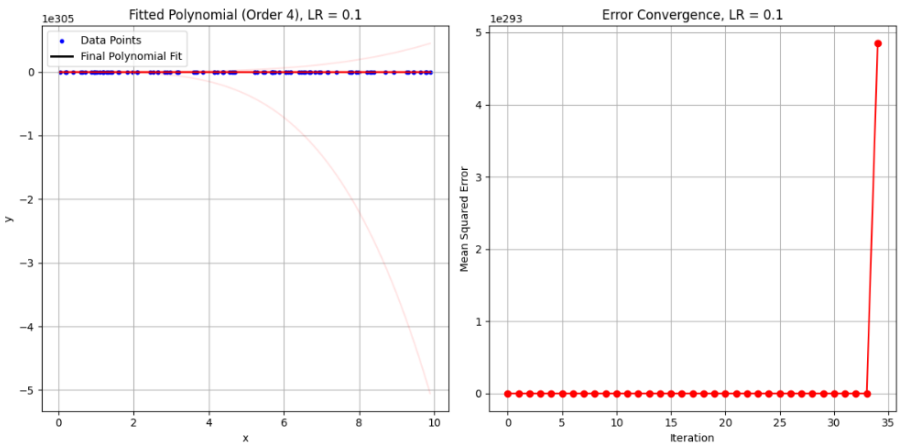



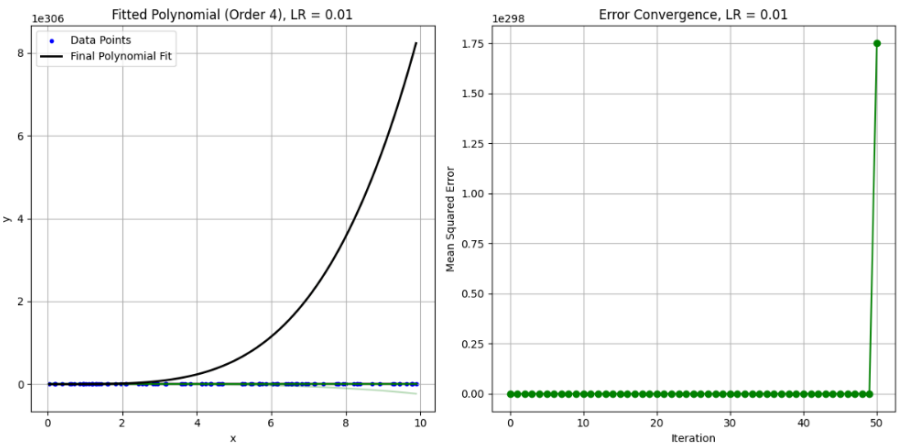


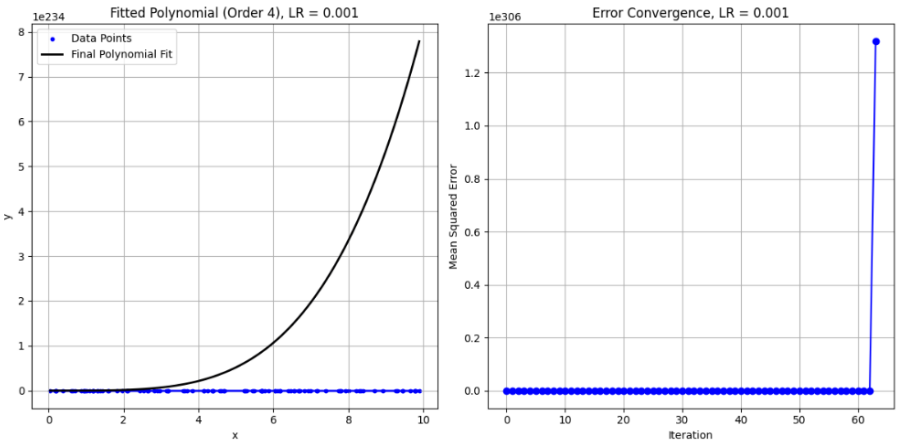

## Question 3

Order 4 polynomial regression. Error + line.

**Answer and Graphs:**




Fitted Polynomial (Order 4), LR = 0.001
1e234
Data Points
Final Polynomial Fit
y
x
Error Convergence, LR = 0.001
1e306
Mean Squared Error
Iteration

# Deliverables

- This report with filled answers and plots.
- Complete Jupyter Notebook implementation (submitted separately).